Azure Data Analytics Case Study

Trip Data Analysis using PySpark on Azure Cloud

1. Introduction

Modern transportation and mobility systems generate large volumes of trip data containing information about travel time, distance, purpose, and location. Analyzing this data can provide valuable insights into travel behavior, operational efficiency, and usage patterns.

Traditional data processing tools struggle to handle large datasets efficiently. Big data technologies enable scalable processing and analysis of such datasets.

This case study focuses on analyzing trip data using **Apache Spark** with PySpark on cloud infrastructure provided by **Microsoft Azure**. The analysis aims to identify travel patterns, evaluate efficiency metrics, detect anomalies, and derive insights from the dataset.

The system uses distributed processing to handle large datasets efficiently and uses Python visualization libraries to present analytical insights.

2. Dataset Description

The dataset used for this case study contains trip-level information including time, distance, category, and purpose of travel.

Column	Description
start_date	Date when the trip started
start_hour	Hour when the trip started
start_minute	Minute when the trip started

end_date	Date when the trip ended
end_hour	Hour when the trip ended
end_minute	Minute when the trip ended
month	Month of the trip
year	Year of the trip
CATEGORY	Category of trip (Business or Personal)
START	Starting location of the trip
STOP	Destination location of the trip
MILES	Distance travelled during the trip
PURPOSE	Purpose of the trip
duration	Total duration of the trip
speed	Average speed during the trip
day	Day of the week
day_time	Time segment of the day (morning, afternoon, evening, night)

This dataset allows analysis of travel patterns, efficiency, behavioral insights, and anomaly detection.

3. System Architecture

The big data analytics workflow implemented in Azure follows the architecture below.

1. The raw dataset is stored in **Azure Data Lake Storage**, which provides scalable storage for large datasets.
2. Data processing and transformation are performed using **Azure Databricks** with **Apache Spark** and PySpark.
3. Spark performs distributed processing to clean, transform, and analyze the dataset.
4. Aggregated results are generated using Spark SQL and PySpark transformations.
5. Analytical insights are visualized using Python libraries such as **Matplotlib** and **Seaborn**.

This architecture allows efficient processing of large-scale datasets and supports scalable analytical workflows.

4. Analytical Use Cases

4.1 Trip Duration Analysis

Trip duration analysis helps understand how much time is typically spent on travel.

Objectives

- Calculate average trip duration
- Identify trips with unusually long or short durations
- Compare duration across different trip categories

Benefits

This analysis helps identify inefficient travel patterns and operational delays.

```python
print("Calculating average, shortest, and longest trip duration by category")

trip_duration_analysis = df_clean.groupBy("CATEGORY").agg(
    F.avg("duration").alias("avg_trip_duration"),
    F.min("duration").alias("shortest_trip_duration"),
    F.max("duration").alias("longest_trip_duration")
).orderBy(F.col("avg_trip_duration").desc())

trip_duration_analysis.show()

print("Top 10 unusual longest trips :")

df_clean.orderBy(F.col("duration").desc()) \
    .select("START", "STOP", "CATEGORY", "MILES", "duration").show(10)
print("Top 10 unusual shortest trips :")

df_clean.orderBy(F.col("duration").asc()) \
    .select("START", "STOP", "CATEGORY", "MILES", "duration") .show(10)
```

```
Calculating average, shortest, and longest trip duration by category
+--------+------------------+----------------------+---------------------+
|CATEGORY| avg_trip_duration|shortest_trip_duration|longest_trip_duration|
+--------+------------------+----------------------+---------------------+
|Business|  23.1534795042898|                   1.0|                330.0|
|Personal|20.466666666666665|                   4.0|                185.0|
+--------+------------------+----------------------+---------------------+
```

```
Top 10 unusual longest trips :
+----------------+----------------+--------+------------------+--------+
|           START|            STOP|CATEGORY|             MILES|duration|
+----------------+----------------+--------+------------------+--------+
|           Latta|    Jacksonville|Business|499.37820200000004|   330.0|
|     Morrisville|      Banner Elk|Business|        314.304102|   206.0|
|           Boone|            Cary|Personal|        290.003068|   185.0|
|       Asheville|          Mebane|Business|        315.269706|   181.0|
|    Jacksonville|       Ridgeland|Business|280.34702799999997|   178.0|
|Unknown Location|Unknown Location|Business|        252.505446|   178.0|
|            Cary|           Latta|Business|         231.74496|   178.0|
|Unknown Location|      R?walpindi|Business|28.807185999999998|   167.0|
|      Rawalpindi|Unknown Location|Business|         165.76202|   166.0|
|       Islamabad|       Islamabad|Business|60.672118000000005|   161.0|
+----------------+----------------+--------+------------------+--------+
only showing top 10 rows
```

```
Top 10 unusual shortest trips :
+---------------+--------------+--------+------------------+--------+
|          START|          STOP|CATEGORY|             MILES|duration|
+---------------+--------------+--------+------------------+--------+
|           Apex|          Apex|Business|           1.60934|     1.0|
|    Whitebridge|Westpark Place|Business|2.5749440000000003|     2.0|
|           Soho|       Tribeca|Business|           0.80467|     2.0|
|         Lahore|        Lahore|Business|2.5749440000000003|     2.0|
|    Whitebridge|Westpark Place|Business|3.5405480000000003|     2.0|
|           Cary|          Cary|Business|           0.80467|     2.0|
|        Colombo|       Colombo|Business|          2.735878|     3.0|
| Kilarney Woods|Kildaire Farms|Business|          2.735878|     3.0|
|           Cary|   Morrisville|Business|          4.02335|     3.0|
|Meredith Townes|  Harden Place|Business|2.2530759999999996|     3.0|
+---------------+--------------+--------+------------------+--------+
only showing top 10 rows
```

4.2 Distance and Speed Analysis

This analysis studies the relationship between travel distance and average speed.

Objectives

- Calculate average speed for different trip categories
- Identify unrealistic speed values
- Evaluate travel efficiency

Benefits

Detects anomalies and helps understand travel performance.

```python
# 1. Average speed & distance
print("\nAverage speed and distance by category:")
df_clean.groupBy("CATEGORY").agg(
    F.avg("MILES").alias("avg_distance"),
    F.avg("speed").alias("avg_speed")
).show()

# 2. Unrealistic speeds
print("\nUnrealistic speed values (>120):")
df_clean.filter(F.col("speed") > 120) .select("START", "STOP", "CATEGORY", "MILES", "duration", "speed").show(10)

# 3. Efficiency
print("\nTravel efficiency (duration per mile):")
df_eff = df_clean.withColumn(
    "duration_per_mile",
    F.col("duration") / F.col("MILES")
)

df_eff.select(F.avg("duration_per_mile"),F.avg("speed")).show()
```

```
Average speed and distance by category:
+--------+------------------+-----------------+
|CATEGORY|      avg_distance|        avg_speed|
+--------+------------------+-----------------+
|Personal|15.155691226666665|33.66036530434003|
|Business|   16.095394415634|37.40244788424314|
+--------+------------------+-----------------+


Unrealistic speed values (>120):
+----------------+----------------+--------+------------------+--------+------------------+
|           START|            STOP|CATEGORY|             MILES|duration|             speed|
+----------------+----------------+--------+------------------+--------+------------------+
| West Palm Beach|      Palm Beach|Business|         11.426314|     5.0|        137.115768|
|Unknown Location|Unknown Location|Business|29.450922000000002|    12.0|         147.25461|
|         Raleigh|            Cary|Business|52.786351999999994|    19.0|166.69374315789472|
|     Morrisville|            Cary|Business|         14.323126|     6.0|         143.23126|
|   San Francisco|         Oakland|Business|         15.932466|     5.0|        191.189592|
|Unknown Location|Unknown Location|Business|         19.794882|     9.0|131.96588000000003|
+----------------+----------------+--------+------------------+--------+------------------+
```

```
Travel efficiency (duration per mile):
+----------------------+-----------------+
|avg(duration_per_mile)|       avg(speed)|
+----------------------+-----------------+
|     2.103733327146251|37.15275376191864|
+----------------------+-----------------+
```

4.3 Time-of-Day Travel Pattern Analysis

Travel activity is analyzed based on the time of day.

Objectives

- Identify peak travel hours
- Analyze trip frequency during different time periods
- Compare travel activity across morning, afternoon, evening, and night

Benefits

Helps understand daily mobility patterns.

```python
from pyspark.sql import functions as F
# 1. Peak travel hours
print("\nIdentifying peak travel hours...")
peak_hours = df_clean.groupBy("start_hour") \
    .count() \
    .orderBy(F.col("count").desc())

peak_hours.show()

# 2. Trip frequency during different time periods
print("\nAnalyzing trip frequency across time periods...")
time_period_analysis = df_clean.groupBy("day_time") \
    .count() \
    .orderBy(F.col("count").desc())
time_period_analysis.show()

# 3. Compare travel activity across morning, afternoon, evening, and night
print("\nComparing travel activity across morning, afternoon, evening, and night...")
df_clean.groupBy("day_time") \
    .agg(
        F.count("*").alias("trip_count"),
        F.avg("MILES").alias("avg_distance"),
        F.avg("duration").alias("avg_duration")
    ).orderBy(F.col("trip_count").desc()).show()
```

```
Identifying peak travel hours...
+----------+-----+
|start_hour|count|
+----------+-----+
|      15.0|   97|
|      17.0|   92|
|      13.0|   92|
|      18.0|   91|
|      14.0|   89|
|      16.0|   88|
|      12.0|   76|
|      11.0|   72|
|      20.0|   71|
|      19.0|   67|
|      10.0|   64|
|       9.0|   50|
|      21.0|   49|
|       8.0|   33|
|      22.0|   30|
|       0.0|   19|
|      23.0|   14|
```

```
Analyzing trip frequency across time periods...
+---------+-----+
| day_time|count|
+---------+-----+
|Afternoon|  549|
|  Morning|  307|
|  Evening|  231|
|    Night|   37|
+---------+-----+


Comparing travel activity across morning, afternoon, evening, and night...
+---------+----------+------------------+----------------+
| day_time|trip_count|      avg_distance|     avg_duration|
+---------+----------+------------------+----------------+
|Afternoon|       549| 16.78579728233154|24.26775956284153|
|  Morning|       307| 15.50575500325733|22.84039087947883|
|  Evening|       231|14.328002787878757| 20.6017316017316|
|    Night|        37|19.873174216216213| 19.7027027027027|
+---------+----------+------------------+----------------+
```

4.4 Day-wise Travel Pattern Analysis

This analysis examines travel patterns across different days of the week.

Objectives

- Identify the busiest days for travel
- Compare weekday and weekend travel behavior
- Measure total distance travelled per day

Benefits

Supports planning and resource allocation.

```python
from pyspark.sql import functions as F
# 1. Identify busiest days
print("\nIdentifying busiest days for travel...")
daywise_analysis = df_clean.groupBy("day").agg(
    F.count("*").alias("trip_count"),
    F.sum("MILES").alias("total_distance"),
    F.avg("duration").alias("avg_duration")
).orderBy(F.col("trip_count").desc())

daywise_analysis.show()
```

(2) Spark Jobs

daywise_analysis: pyspark.sql.dataframe.DataFrame = [day: string, trip_count: long ... 2 more fields]

```
Identifying busiest days for travel...
+---------+----------+------------------+------------------+
|      day|trip_count|    total_distance|      avg_duration|
+---------+----------+------------------+------------------+
|   Friday|       200| 3737.531215999995|            24.835|
|   Monday|       173| 2215.900245999997|20.040462427745666|
|  Tuesday|       170|1894.9978499999984|              20.5|
| Thursday|       149|2655.2500659999973|24.966442953020135|
|Wednesday|       144|1953.7387599999984| 21.34722222222222|
| Saturday|       144|2893.9151879999968|            26.8125|
|   Sunday|       144|2669.4122579999985|            22.5625|
+---------+----------+------------------+------------------+
```

```python
print("\nComparing weekday vs weekend travel behavior...")

df_day_type = df_clean.withColumn(
    "day_type",
    F.when(F.col("day").isin("Saturday", "Sunday"), "Weekend")
     .otherwise("Weekday")
)

df_day_type.groupBy("day_type").agg(
    F.count("*").alias("trip_count"),
    F.sum("MILES").alias("total_distance"),
    F.avg("duration").alias("avg_duration")
).show()
```

(2) Spark Jobs

df_day_type: pyspark.sql.dataframe.DataFrame = [start_date: date, start_hour: double ... 16 more fields]

```
Comparing weekday vs weekend travel behavior...
+--------+----------+------------------+----------------+
|day_type|trip_count|    total_distance|     avg_duration|
+--------+----------+------------------+----------------+
| Weekday|       836|12457.418138000048|22.38397129186603|
| Weekend|       288| 5563.327446000006|          24.6875|
+--------+----------+------------------+----------------+
```

4.5 Category-Based Trip Analysis

Trips are analyzed based on category to identify behavioral differences.

Objectives

- Compare business and personal travel
- Analyze average distance and duration for each category
- Identify dominant travel purposes

Benefits

Provides insights into travel behavior.

```
from pyspark.sql import functions as F
# 1. Compare Business vs Personal + distance & duration
print("Comparing Business and Personal travel with distance and duration:")

category_analysis = df_clean.groupBy("CATEGORY").agg(
    F.count("*").alias("trip_count"),
    F.avg("MILES").alias("avg_distance"),
    F.avg("duration").alias("avg_duration")
).orderBy(F.col("trip_count").desc())

category_analysis.show()

# 2. Identify dominant travel purposes
print("\nIdentifying dominant travel purposes:")

purpose_analysis = df_clean.filter(F.col("PURPOSE").isNotNull()) \
    .groupBy("CATEGORY", "PURPOSE") \
    .agg(F.count("*").alias("purpose_count")) \
    .orderBy(F.col("purpose_count").desc())
```

```
Comparing Business and Personal travel with distance and duration:
+--------+----------+------------------+------------------+
|CATEGORY|trip_count|      avg_distance|      avg_duration|
+--------+----------+------------------+------------------+
|Business|      1049|   16.095394415634|  23.1534795042898|
|Personal|        75|15.155691226666665|20.466666666666665|
+--------+----------+------------------+------------------+
```

```
Identifying dominant travel purposes:
+--------+--------------+-------------+
|CATEGORY|       PURPOSE|purpose_count|
+--------+--------------+-------------+
|Business|       Meeting|          278|
|Business| Meal/Entertain|          227|
|Business| Temporary Site|          170|
|Business|Errand/Supplies|          159|
|Business| Customer Visit|          143|
|Business| Airport/Travel|           54|
|Business|Between Offices|           18|
|Personal| Meal/Entertain|           18|
|Personal|       Meeting|           16|
|Personal|Errand/Supplies|           14|
+--------+--------------+-------------+
only showing top 10 rows
```

4.6 Monthly Travel Trend Analysis

Travel data is analyzed across months to identify seasonal patterns.

Objectives

- Calculate total trips per month
- Identify months with highest travel activity
- Analyze trends across years

Benefits

Helps identify seasonal demand.

```python
from pyspark.sql import functions as F
# 1. Total trips per month (with year)
print("\nTotal trips per month:")

monthly_trips = df_clean.groupBy("year", "month").agg(
    F.count("*").alias("trip_count"),
    F.sum("MILES").alias("total_distance"),
    F.avg("duration").alias("avg_duration")
).orderBy("year", "month")

monthly_trips.show()

# 2. Months with highest travel activity
print("\nTop months with highest travel activity:")

monthly_trips.orderBy(F.col("trip_count").desc()).select("year", "month", "trip_count").show(5)

# 3. Year-wise trend analysis
print("\nYear-wise travel trend:")

yearly_trend = df_clean.groupBy("year").agg(
    F.count("*").alias("total_trips"),F.sum("MILES").alias("total_distance")
).orderBy("year")

yearly_trend.show()
```

```
Total trips per month:
+----+-----+----------+------------------+------------------+
|year|month|trip_count|    total_distance|      avg_duration|
+----+-----+----------+------------------+------------------+
|2016|April|        53|1559.4504600000005| 29.81132075471698|
|2016|  Aug|       133|2149.2735699999994|24.984962406015036|
|2016|  Dec|       146|1579.2453419999986| 21.36986301369863|
|2016|  Feb|       113|1419.4378799999997|19.831858407079647|
|2016|  Jan|        61| 825.4304859999997|18.295081967213115|
|2016| July|       109|1924.9315739999993| 21.55045871559633|
|2016| June|       103|1276.6894219999992|20.106796116504853|
|2016|  Mar|       112|2402.5836859999986|              24.5|
|2016|  May|        47| 552.4864220000001|14.914893617021276|
|2016|  Nov|       122|1314.6698459999989| 17.68032786885246|
|2016|  Oct|        96|2530.6871500000007|32.364583333333336|
|2016|  Sep|        29| 485.8597459999999|45.310344827586206|
+----+-----+----------+------------------+------------------+
```

```
Top months with highest travel activity:
+----+-----+----------+
|year|month|trip_count|
+----+-----+----------+
|2016|  Dec|       146|
|2016|  Aug|       133|
|2016|  Nov|       122|
|2016|  Feb|       113|
|2016|  Mar|       112|
+----+-----+----------+
only showing top 5 rows

Year-wise travel trend:
+----+-----------+------------------+
|year|total_trips|    total_distance|
+----+-----------+------------------+
|2016|       1124|18020.745584000088|
+----+-----------+------------------+
```

4.7 Travel Efficiency Analysis

Efficiency analysis measures how effectively travel resources are utilized.

Metrics

- Miles travelled per trip
- Duration per mile
- Average speed

Benefits

Helps identify inefficient travel patterns and potential improvements.

```python
print("\nCalculating travel efficiency metrics...")
travel_efficiency = df_clean.withColumn(
    "duration_per_mile",
    F.col("duration") / F.col("MILES")
)

travel_efficiency.groupBy("CATEGORY").agg(
    F.count("*").alias("trip_count"),
    F.avg("MILES").alias("avg_miles_per_trip"),
    F.avg("duration_per_mile").alias("avg_duration_per_mile"),
    F.avg("speed").alias("avg_speed")
).show()

# Identify inefficient trips
print("\nIdentifying least efficient trips (high duration per mile)...")
travel_efficiency.orderBy(F.col("duration_per_mile").desc()) \
    .select("START", "STOP", "CATEGORY", "MILES", "duration", "speed", "duration_per_mile").show(10)
```

```
Calculating travel efficiency metrics...
+--------+----------+------------------+---------------------+-----------------+
|CATEGORY|trip_count|avg_miles_per_trip|avg_duration_per_mile|        avg_speed|
+--------+----------+------------------+---------------------+-----------------+
|Personal|        75|15.155691226666665|    2.147495444688436|33.66036530434003|
|Business|      1049|   16.095394415634|    2.100604481754766|37.40244788424314|
+--------+----------+------------------+---------------------+-----------------+
```

```
Identifying least efficient trips (high duration per mile)...
+--------------------+----------------+--------+------------------+--------+------------------+------------------+
|               START|            STOP|CATEGORY|             MILES|duration|             speed| duration_per_mile|
+--------------------+----------------+--------+------------------+--------+------------------+------------------+
|    Unknown Location|Unknown Location|Business|         12.713786|   121.0| 6.304356694214877| 9.517227991724887|
|    Unknown Location|      R?walpindi|Business|15.449663999999999|   120.0| 7.724831999999999| 7.767159208122585|
|    Unknown Location|       Islamabad|Business|         20.277684|   154.0| 7.900396363636363| 7.594555670164304|
|              Lahore|Unknown Location|Business|         11.748182|    80.0|         8.8111365| 6.809564237258156|
|             Kar?chi|         Kar?chi|Business|1.1265379999999998|     7.0| 9.656039999999999| 6.213727366498069|
|          Emeryville|        Berkeley|Business|          6.276426|    39.0| 9.656039999999999| 6.213727366498068|
|     Jamestown Court| Jamestown Court|Business|           1.60934|    10.0|           9.65604| 6.213727366498068|
|             Central|         Central|Business|          4.184284|    26.0| 9.656039999999999| 6.213727366498068|
|          Katunayaka|      Katunayaka|Business|           0.80467|     5.0|           9.65604| 6.213727366498068|
|Lexington Park at...|  Westpark Place|Business|          3.057746|    18.0|10.192486666666667|5.8866890840508015|
+--------------------+----------------+--------+------------------+--------+------------------+------------------+
only showing top 10 rows
```

4.8 Peak Travel Period Detection

This analysis identifies periods with the highest travel demand.

Objectives

- Count trips per hour
- Identify peak travel windows
- Analyze travel distribution across time segments

Benefits

Supports demand forecasting and traffic analysis.

```python
# 1. Count trips per hour
print("\nCounting trips per hour...")
hourly_trips = df_clean.groupBy("start_hour").agg(
    F.count("*").alias("trip_count")
).orderBy(F.col("trip_count").desc())

hourly_trips.show()


# 2. Identify peak travel windows
print("\nIdentifying peak travel windows...")
hourly_trips.show(5)

# 3. Analyze travel distribution across time segments
print("\nAnalyzing travel distribution across time segments...")

time_segment_analysis = df_clean.groupBy("day_time").agg(
    F.count("*").alias("trip_count"),
    F.avg("MILES").alias("avg_distance"),
    F.avg("duration").alias("avg_duration")
).orderBy(F.col("trip_count").desc())

time_segment_analysis.show()
```

```
Counting trips per hour...
+----------+----------+
|start_hour|trip_count|
+----------+----------+
|      15.0|        97|
|      17.0|        92|
|      13.0|        92|
|      18.0|        91|
|      14.0|        89|
|      16.0|        88|
|      12.0|        76|
|      11.0|        72|
|      20.0|        71|
|      19.0|        67|
|      10.0|        64|
|       9.0|        50|
|      21.0|        49|
|       8.0|        33|
|      22.0|        30|
```

```
Identifying peak travel windows...
+----------+----------+
|start_hour|trip_count|
+----------+----------+
|      15.0|        97|
|      13.0|        92|
|      17.0|        92|
|      18.0|        91|
|      14.0|        89|
+----------+----------+
only showing top 5 rows

Analyzing travel distribution across time segments...
+---------+----------+------------------+-----------------+
| day_time|trip_count|      avg_distance|     avg_duration|
+---------+----------+------------------+-----------------+
|Afternoon|       549| 16.78579728233154|24.26775956284153|
|  Morning|       307| 15.50575500325733|22.84039087947883|
|  Evening|       231|14.328002787878757| 20.6017316017316|
|    Night|        37|19.873174216216213| 19.7027027027027|
+---------+----------+------------------+-----------------+
```

4.9 Long Distance Trip Identification

This analysis identifies trips that involve significant distance or time.

Objectives

- Detect trips exceeding predefined distance thresholds
- Identify trips with unusually long duration
- Analyze high-resource travel patterns

Benefits

Helps evaluate travel costs and logistics.

```python
distance_threshold = 50

print("\nDetecting trips exceeding predefined distance threshold...")
long_distance_trips = df_clean.filter(
    F.col("MILES") > distance_threshold
).select(
    "START", "STOP", "CATEGORY", "MILES", "duration", "speed", "PURPOSE"
).orderBy(F.col("MILES").desc())
long_distance_trips.show(10)


print("\nIdentifying trips with unusually long duration...")
long_duration_trips = df_clean.orderBy(F.col("duration").desc()) \
    .select("START", "STOP", "CATEGORY", "MILES", "duration", "speed", "PURPOSE")
long_duration_trips.show(10)

print("\nAnalyzing high-resource travel patterns...")
high_resource_analysis = df_clean.filter(
    (F.col("MILES") > distance_threshold) | (F.col("duration") > 60)
).groupBy("CATEGORY").agg(
    F.count("*").alias("trip_count"),
    F.avg("MILES").alias("avg_distance"),
    F.avg("duration").alias("avg_duration"),
    F.avg("speed").alias("avg_speed")
)
high_resource_analysis.show()
```

```
Detecting trips exceeding predefined distance threshold...
+----------------+----------------+--------+------------------+--------+------------------+----------------+
|           START|            STOP|CATEGORY|             MILES|duration|             speed|         PURPOSE|
+----------------+----------------+--------+------------------+--------+------------------+----------------+
|           Latta|    Jacksonville|Business|499.37820200000004|   330.0| 90.79603672727274| Customer Visit|
|       Asheville|          Mebane|Business|        315.269706|   181.0|104.50929480662982|         Meeting|
|     Morrisville|      Banner Elk|Business|        314.304102|   206.0|  91.5448840776699|Errand/Supplies|
|           Boone|            Cary|Personal|        290.003068|   185.0| 94.05504908108107|         Commute|
|    Jacksonville|       Ridgeland|Business|280.34702799999997|   178.0| 94.49899820224718| Customer Visit|
|        Florence|            Cary|Business|        256.367862|   136.0|113.10346852941177|         Meeting|
|Unknown Location|Unknown Location|Business|        252.505446|   178.0| 85.11419528089887| Airport/Travel|
|            Cary|           Latta|Business|         231.74496|   178.0| 78.11627865168539| Customer Visit|
|          Austin|            Katy|Business|         218.87024|   139.0| 94.47636258992804| Customer Visit|
|   Winston Salem|       Asheville|Business|        215.007824|   114.0|113.16201263157895|         Meeting|
+----------------+----------------+--------+------------------+--------+------------------+----------------+
```

```
Identifying trips with unusually long duration...
+----------------+----------------+--------+------------------+--------+------------------+----------------+
|           START|            STOP|CATEGORY|             MILES|duration|             speed|         PURPOSE|
+----------------+----------------+--------+------------------+--------+------------------+----------------+
|           Latta|    Jacksonville|Business|499.37820200000004|   330.0| 90.79603672727274| Customer Visit|
|     Morrisville|      Banner Elk|Business|        314.304102|   206.0|  91.5448840776699|Errand/Supplies|
|           Boone|            Cary|Personal|        290.003068|   185.0| 94.05504908108107|         Commute|
|       Asheville|          Mebane|Business|        315.269706|   181.0|104.50929480662982|         Meeting|
|    Jacksonville|       Ridgeland|Business|280.34702799999997|   178.0| 94.49899820224718| Customer Visit|
|Unknown Location|Unknown Location|Business|        252.505446|   178.0| 85.11419528089887| Airport/Travel|
|            Cary|           Latta|Business|         231.74496|   178.0| 78.11627865168539| Customer Visit|
|Unknown Location|      R?walpindi|Business|28.807185999999998|   167.0|10.349887185628742| Temporary Site|
|      Rawalpindi|Unknown Location|Business|         165.76202|   166.0| 59.91398313253013|         Meeting|
|       Islamabad|       Islamabad|Business|60.672118000000005|   161.0| 22.61072720496895| Temporary Site|
+----------------+----------------+--------+------------------+--------+------------------+----------------+
```

```
Analyzing high-resource travel patterns...
+--------+----------+------------------+----------------+----------------+
|CATEGORY|trip_count|      avg_distance|     avg_duration|        avg_speed|
+--------+----------+------------------+----------------+----------------+
|Personal|         2|178.71720699999997|           136.5|70.01548340417689|
|Business|        60| 99.72007086666669|99.66666666666667|59.58120672259382|
+--------+----------+------------------+----------------+----------------+
```

4.10 Data Quality Monitoring

This analysis ensures the dataset is clean and reliable.

Possible Data Issues

- Negative trip duration
- Missing values
- Unrealistic speeds
- Trips with zero distance but long duration
- End time earlier than start time

Benefits

Improves reliability of analytical results.

```python
print("\nChecking missing values...")
df_quality.select([
    F.count(F.when(F.col(c).isNull(), c)).alias(c)
    for c in df_quality.columns
]).show()
print("\nChecking negative trip duration...")
df_quality.filter(F.col("duration") < 0) \
    .select("START", "STOP", "MILES", "duration").show(10)
print("\nChecking unrealistic speed values...")
df_quality.filter((F.col("speed") > 120) | (F.col("speed") <= 0)) \
    .select("START", "STOP", "MILES", "duration", "speed") \
    .show(10)

print("\nChecking zero distance but long duration...")
df_quality.filter((F.col("MILES") == 0) & (F.col("duration") > 30)) \
    .select("START", "STOP", "MILES", "duration").show(10)


print("\nChecking invalid time sequence (end < start)...")
df_quality.filter(
    (F.col("end_hour") < F.col("start_hour")) |
    ((F.col("end_hour") == F.col("start_hour")) &
     (F.col("end_minute") < F.col("start_minute")))
).select( "start_hour", "start_minute", "end_hour", "end_minute").show(10)
```

```
Checking missing values...
+----------+----------+------------+--------+--------+----------+-----+----+--------+-----+----+-----+-------+--------+----
-+---+--------+
|start_date|start_hour|start_minute|end_date|end_hour|end_minute|month|year|CATEGORY|START|STOP|MILES|PURPOSE|duration|spee
d|day|day_time|
+----------+----------+------------+--------+--------+----------+-----+----+--------+-----+----+-----+-------+--------+----
-+---+--------+
|         0|         0|           0|       0|       0|         0|    0|   0|       0|    0|   0|    0|      0|       0|
0|  0|       0|
+----------+----------+------------+--------+--------+----------+-----+----+--------+-----+----+-----+-------+--------+----
-+---+--------+


Checking negative trip duration...
+-----+----+-----+--------+
|START|STOP|MILES|duration|
+-----+----+-----+--------+
+-----+----+-----+--------+
```

```
Checking unrealistic speed values...
+----------------+----------------+------------------+--------+------------------+
|           START|            STOP|             MILES|duration|             speed|
+----------------+----------------+------------------+--------+------------------+
| West Palm Beach|      Palm Beach|         11.426314|     5.0|        137.115768|
|Unknown Location|Unknown Location|          37.81949|     8.0|        283.646175|
|Unknown Location|Unknown Location|29.450922000000002|    12.0|         147.25461|
|         Raleigh|            Cary|52.786351999999994|    19.0|166.69374315789472|
|     Morrisville|         Raleigh|         12.230984|     2.0|366.92951999999997|
|     Morrisville|            Cary|         14.323126|     6.0|         143.23126|
|Unknown Location|Unknown Location|111.20539399999998|     0.0|          Infinity|
|Unknown Location|Unknown Location|13.840323999999999|     4.0|207.60485999999997|
|Unknown Location|Unknown Location|         15.771532|     2.0|         473.14596|
|Unknown Location|Unknown Location|2.5749440000000003|     0.0|          Infinity|
+----------------+----------------+------------------+--------+------------------+
only showing top 10 rows
```

```
Checking zero distance but long duration...
+-----+----+-----+--------+
|START|STOP|MILES|duration|
+-----+----+-----+--------+
+-----+----+-----+--------+


Checking invalid time sequence (end < start)...
+----------+------------+--------+----------+
|start_hour|start_minute|end_hour|end_minute|
+----------+------------+--------+----------+
+----------+------------+--------+----------+
```

4.11 Travel Behavior Segmentation

Machine learning methods can be applied to identify clusters of similar trips.

Features Used

- Distance
- Duration
- Speed
- Time of day

Clustering algorithms can group trips with similar patterns.

Benefits

Helps identify distinct travel behaviors.

```python
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.clustering import KMeans

segmentation_df = df_clean.withColumn("MILES", F.expr("try_cast(MILES as double)")) \
                          .withColumn("duration", F.expr("try_cast(duration as double)")) \
                          .withColumn("speed", F.expr("try_cast(speed as double)"))

day_time_indexer = StringIndexer(inputCol="day_time", outputCol="day_time_index")
segmentation_df = day_time_indexer.fit(segmentation_df).transform(segmentation_df)

feature_df = segmentation_df.select(
    "START", "STOP", "CATEGORY", "PURPOSE", "MILES", "duration", "speed", "day_time", "day_time_index").dropna()

assembler = VectorAssembler(inputCols=["MILES", "duration", "speed", "day_time_index"],
    outputCol="features")
feature_data = assembler.transform(feature_df)

kmeans = KMeans(featuresCol="features", predictionCol="cluster", k=3, seed=42)
model = kmeans.fit(feature_data)
clustered_data = model.transform(feature_data)

print("\nClustered travel behaviors:")
clustered_data.select("START", "STOP", "CATEGORY", "PURPOSE", "MILES", "duration", "speed", "day_time", "cluster"
).show(20)

print("\nCluster summary:")
clustered_data.groupBy("cluster").agg(
    F.count("*").alias("trip_count"),
    F.avg("MILES").alias("avg_distance"),
    F.avg("duration").alias("avg_duration"),
    F.avg("speed").alias("avg_speed")
).orderBy("cluster").show()
```

```
Clustered travel behaviors:
+----------------+---------------+--------+---------------+------------------+--------+------------------+---------+-------+
|           START|           STOP|CATEGORY|        PURPOSE|             MILES|duration|             speed| day_time|cluster|
+----------------+---------------+--------+---------------+------------------+--------+------------------+---------+-------+
|      Fort Pierce|    Fort Pierce|Business| Meal/Entertain| 8.207633999999999|     6.0| 82.07633999999999|  Evening|      0|
|      Fort Pierce|    Fort Pierce|Business| Meal/Entertain|            8.0467|    12.0| 40.23349999999999|    Night|      0|
|      Fort Pierce|    Fort Pierce|Business|Errand/Supplies| 7.724831999999999|    13.0|35.653070769230766|  Evening|      0|
|      Fort Pierce|    Fort Pierce|Business|        Meeting|          7.563898|    14.0| 32.41670571428571|Afternoon|      0|
|      Fort Pierce|West Palm Beach|Business| Customer Visit|102.51495800000001|    67.0|          91.80444|Afternoon|      0|
|  West Palm Beach|West Palm Beach|Business| Meal/Entertain|6.9201619999999995|     4.0|103.80242999999999|Afternoon|      0|
|  West Palm Beach|     Palm Beach|Business|        Meeting|         11.426314|     5.0|        137.115768|Afternoon|      0|
|             Cary|           Cary|Business|        Meeting|1.2874720000000002|     6.0|12.874720000000002|Afternoon|      0|
|             Cary|    Morrisville|Business|        Meeting|13.357522000000001|    20.0| 40.07256600000001|  Morning|      0|
|          Jamaica|       New York|Business| Customer Visit|          26.55411|    27.0| 59.00913333333334|  Morning|      0|
|         New York|         Queens|Business|        Meeting|         17.380872|    43.0|24.252379534883723|Afternoon|      0|
|         Elmhurst|       New York|Business|        Meeting|          12.07005|    35.0|20.691514285714284|Afternoon|      0|
|          Midtown|    East Harlem|Business|        Meeting| 9.977908000000001|    20.0|29.933724000000005|  Evening|      0|
|      East Harlem|          NoMad|Business| Temporary Site|10.299776000000001|    26.0|23.768713846153847|  Morning|      0|
|Flatiron District|        Midtown|Business|Errand/Supplies|2.5749440000000003|     7.0|22.070948571428573|  Morning|      0|
|          Midtown|   Midtown East|Business| Meal/Entertain|          2.735878|    14.0|11.725191428571428|Afternoon|      0|
|     Midtown East|        Midtown|Business| Meal/Entertain|          3.057746|    13.0|14.112673846153845|Afternoon|      0|
|          Midtown|  Hudson Square|Business| Meal/Entertain|          3.057746|    16.0|11.466547499999999|  Morning|      0|
|    Hudson Square|Lower Manhattan|Business| Meal/Entertain|           6.43736|    16.0|           24.1401|  Morning|      0|
|  Lower Manhattan|  Hudson Square|Business|Errand/Supplies|          2.896812|    14.0|12.414908571428573|Afternoon|      0|
+----------------+---------------+--------+---------------+------------------+--------+------------------+---------+-------+
only showing top 20 rows
```

```
Cluster summary:
+-------+----------+------------------+------------------+-----------------+
|cluster|trip_count|      avg_distance|      avg_duration|        avg_speed|
+-------+----------+------------------+------------------+-----------------+
|      0|      1105|12.562173064253443|20.755656108597286|36.30976965080419|
|      1|         1|499.37820200000004|             330.0|90.79603672727274|
|      2|        18|202.23145255555556|142.11111111111111|85.92242930725737|
+-------+----------+------------------+------------------+-----------------+
```

4.12 Travel Duration Prediction

Predictive models can estimate travel time based on historical data.

Features

- Distance travelled
- Time of day
- Day of the week
- Trip category

Regression models can be trained using Spark machine learning libraries.

Benefits

Improves planning and travel time estimation.

```
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator
prediction_df = df_clean.withColumn("MILES", F.expr("try_cast(MILES as double)")) \
                        .withColumn("duration", F.expr("try_cast(duration as double)"))

day_time_indexer = StringIndexer(inputCol="day_time", outputCol="day_time_index", handleInvalid="keep")
day_indexer = StringIndexer(inputCol="day", outputCol="day_index", handleInvalid="keep")
category_indexer = StringIndexer(inputCol="CATEGORY", outputCol="category_index", handleInvalid="keep")

prediction_df = day_time_indexer.fit(prediction_df).transform(prediction_df)
prediction_df = day_indexer.fit(prediction_df).transform(prediction_df)
prediction_df = category_indexer.fit(prediction_df).transform(prediction_df)

feature_df = prediction_df.select("MILES", "day_time_index", "day_index", "category_index", "duration").dropna()
assembler = VectorAssembler(inputCols=["MILES", "day_time_index", "day_index", "category_index"],outputCol="features")

model_data = assembler.transform(feature_df).select("features", "duration")

train_data, test_data = model_data.randomSplit([0.8, 0.2], seed=42)
lr = LinearRegression(featuresCol="features", labelCol="duration")
lr_model = lr.fit(train_data)
predictions = lr_model.transform(test_data)
print("\nSample predicted durations:")
predictions.select("duration", "prediction").show(10)
rmse_evaluator = RegressionEvaluator(labelCol="duration", predictionCol="prediction", metricName="rmse")
r2_evaluator = RegressionEvaluator(labelCol="duration", predictionCol="prediction", metricName="r2")

rmse = rmse_evaluator.evaluate(predictions)
r2 = r2_evaluator.evaluate(predictions)
print(f"RMSE: {rmse}")
print(f"R2 Score: {r2}")
```

```
Sample predicted durations:
+--------+------------------+
|duration|        prediction|
+--------+------------------+
|     7.0|14.846627445787822|
|     8.0|15.389827701796191|
|     3.0|15.498467752997865|
|    13.0|16.584868265014602|
|    15.0|16.910788418619624|
|     7.0| 17.56262872582967|
|    15.0|19.192229493854775|
|    53.0|21.147750415484904|
|    28.0|22.994631285913357|
|    70.0|23.863751695526744|
+--------+------------------+
only showing top 10 rows
RMSE: 12.031655037831904
R2 Score: 0.5639525475878824
```

5. Tools and Technologies

The following technologies are used in this big data analytics workflow:

- **Microsoft Azure**
- **Azure Data Lake Storage**
- **Azure Databricks**
- **Apache Spark**
- **PySpark**
- **Python**
- **Matplotlib**
- **Seaborn**

These tools enable scalable storage, distributed processing, and effective data visualization.

6. Visualization and Reporting

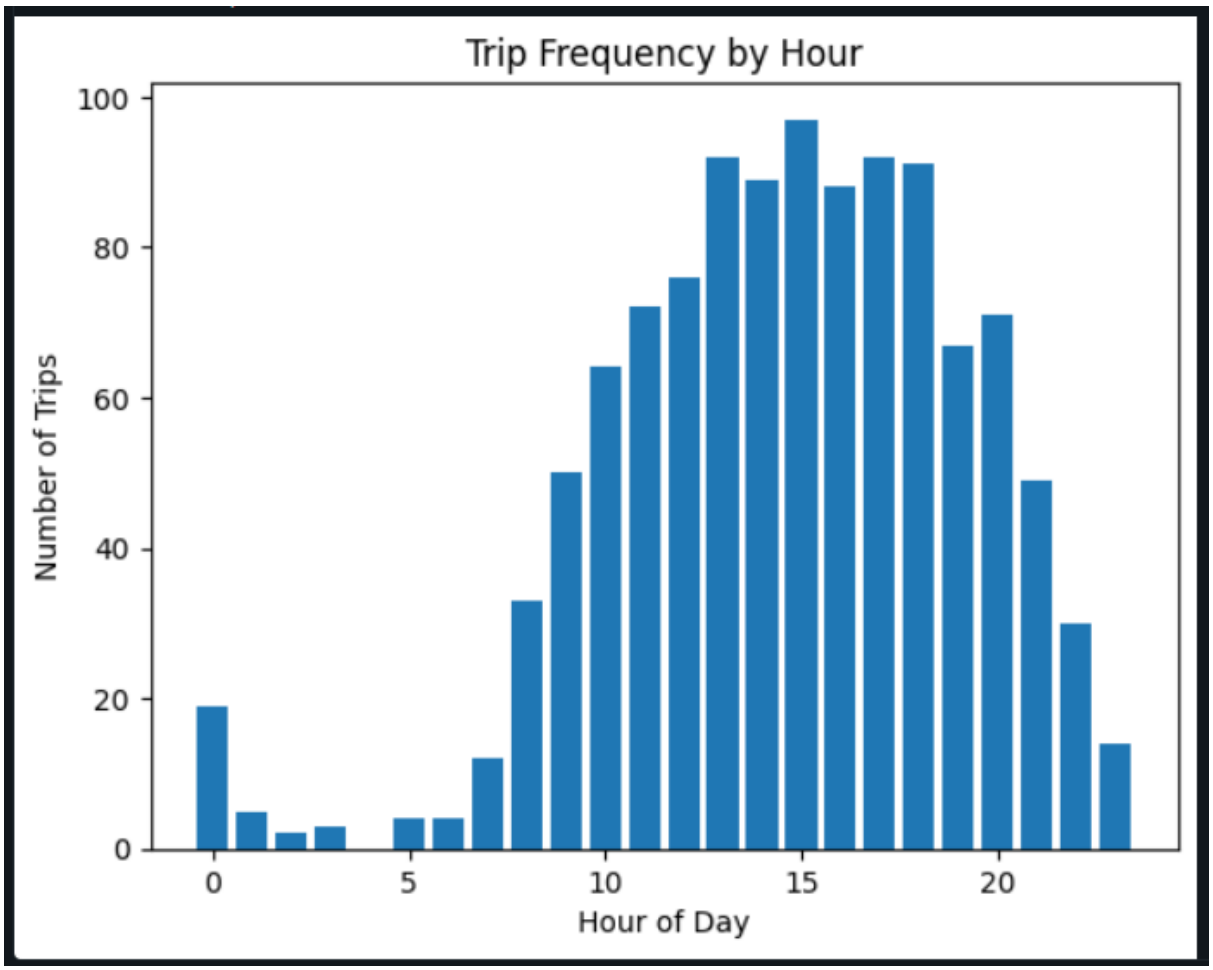
After processing the dataset using PySpark, the analytical results are visualized using Python libraries such as Matplotlib and Seaborn.

Since PySpark processes large volumes of data, aggregated results are first computed using Spark transformations. These results are then converted into Pandas DataFrames for visualization.

Key Visualizations

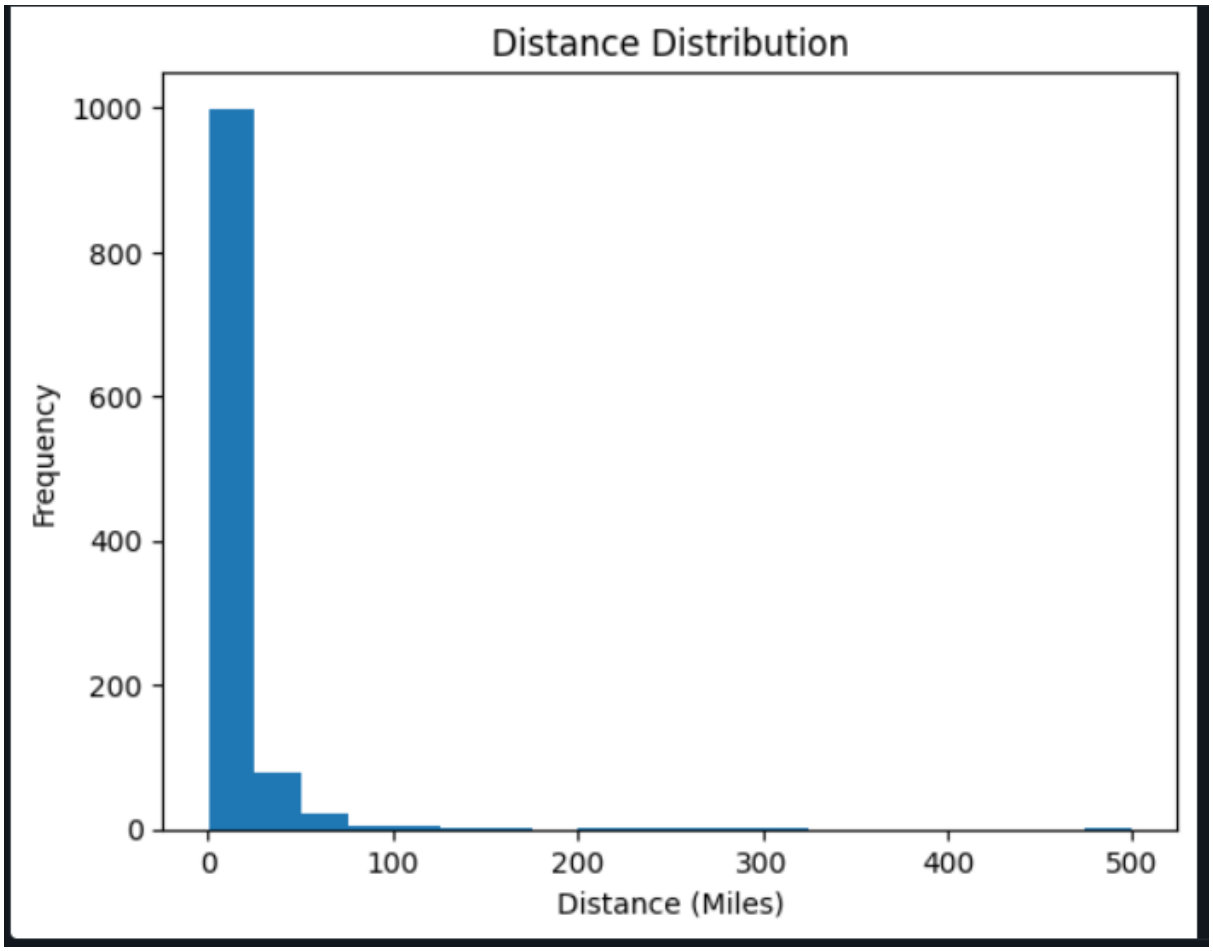
Trip Frequency by Hour
Displays the number of trips occurring during each hour of the day.

```
import matplotlib.pyplot as plt

hour_df = df_clean.groupBy("start_hour").count().orderBy("start_hour").toPandas()

plt.figure()
plt.bar(hour_df["start_hour"], hour_df["count"])
plt.title("Trip Frequency by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Trips")
plt.show()
```



Distance Distribution
Shows how trip distances are distributed across the dataset.

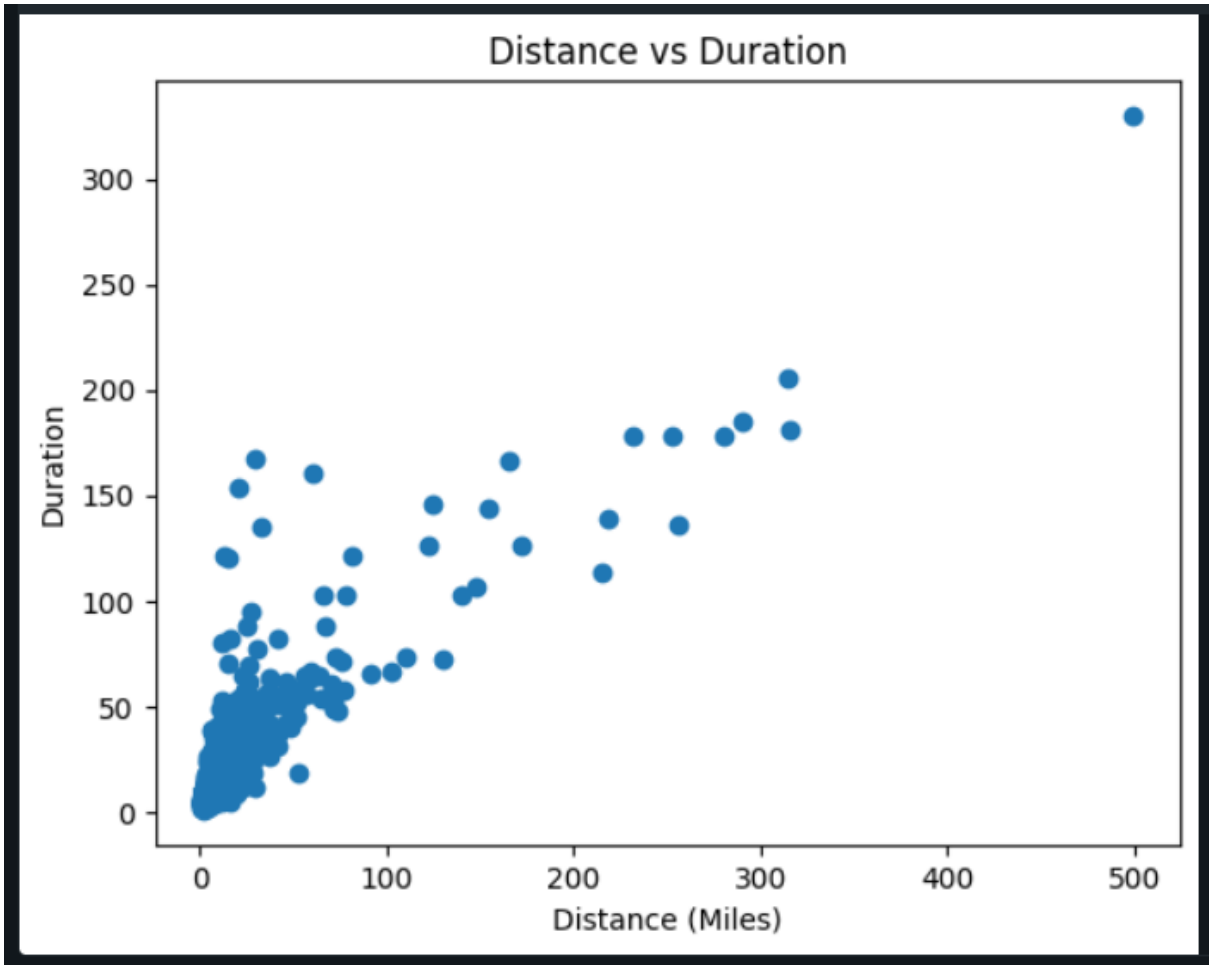
```
plt.figure()
plt.hist(pdf["MILES"], bins=20)
plt.title("Distance Distribution")
plt.xlabel("Distance (Miles)")
plt.ylabel("Frequency")
plt.show()
```



Distance vs Duration Relationship

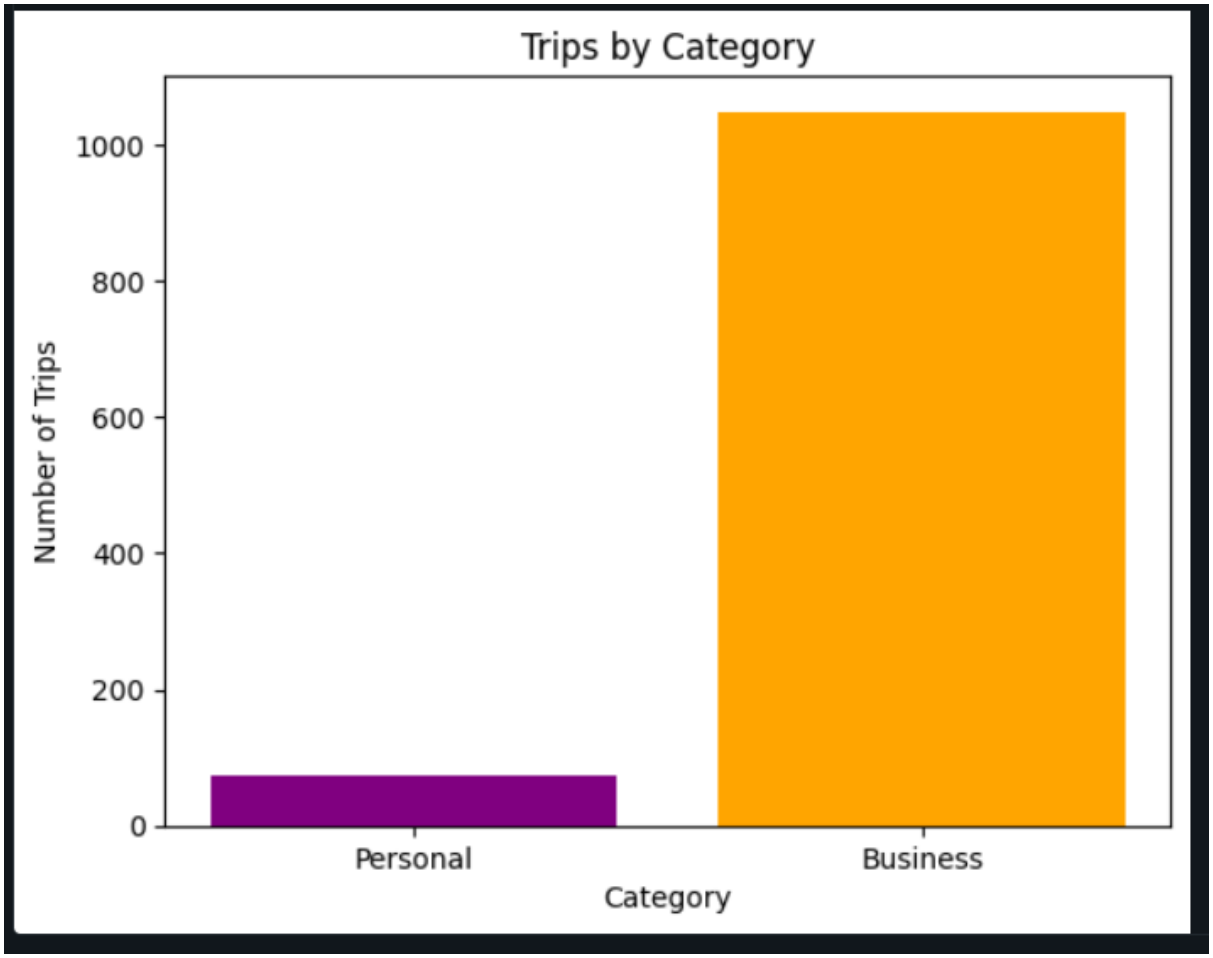
A scatter plot that illustrates how trip duration changes with travel distance.

```
plt.figure()
plt.scatter(pdf["MILES"], pdf["duration"])
plt.title("Distance vs Duration")
plt.xlabel("Distance (Miles)")
plt.ylabel("Duration")
plt.show()
```



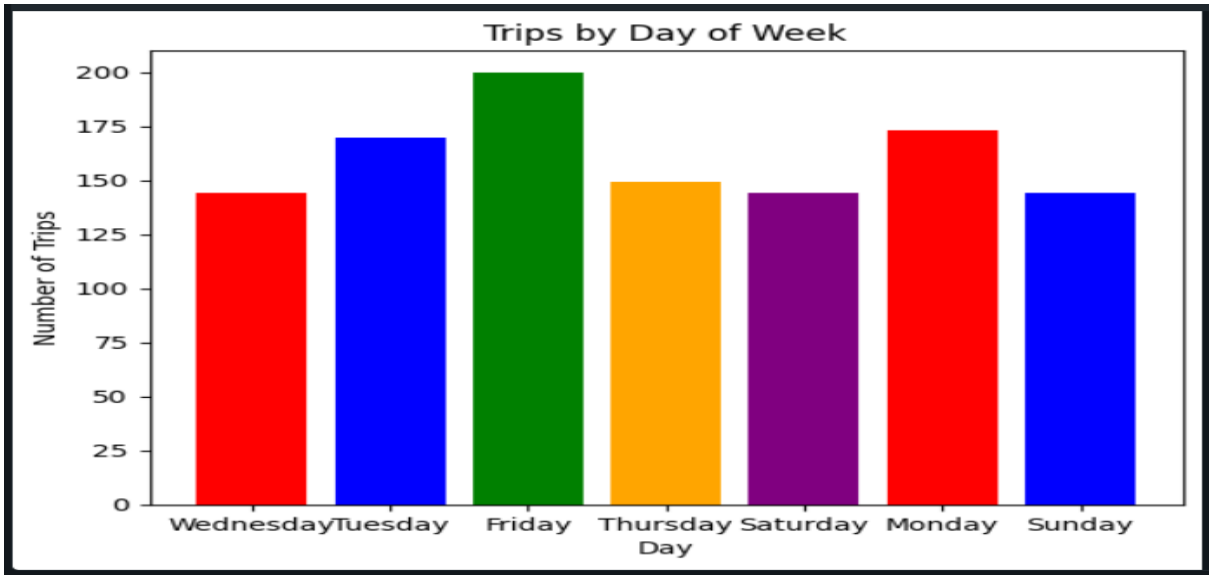
Trips by Category
Compares trip counts between business and personal travel.

```
category_df = df_clean.groupBy("CATEGORY").count().toPandas()

plt.figure()
plt.bar(category_df["CATEGORY"], category_df["count"],color=["purple","orange"])
plt.title("Trips by Category")
plt.xlabel("Category")
plt.ylabel("Number of Trips")
plt.show()
```



Trips by Day of the Week
Shows travel patterns across different days.

```
colors = ["red", "blue", "green", "orange", "purple"]
day_df = df_clean.groupBy("day").count().toPandas()
plt.bar(day_df["day"], day_df["count"], color=colors)
plt.title("Trips by Day of Week")
plt.xlabel("Day")
plt.ylabel("Number of Trips")
plt.show()
```



These visualizations help convert analytical outputs into easily interpretable insights.

7. Expected Outcomes

The implementation of this analytical framework produces several valuable outcomes:

- Identification of peak travel hours
- Understanding of travel behavior patterns
- Detection of data anomalies
- Evaluation of travel efficiency
- Insights into trip categories and purposes
- Predictive capability for estimating travel duration

These insights support data-driven decision making.

8. Conclusion

Big data analytics using PySpark on Azure provides a scalable platform for processing and analyzing large datasets. By applying distributed data processing techniques, the system can efficiently analyze travel patterns, identify inefficiencies, and detect anomalies.

The integration of Azure cloud infrastructure with Spark processing and Python visualization tools enables efficient data analysis and reporting. This approach allows organizations to transform raw trip data into actionable insights that support operational improvements and strategic planning.